ResolveOps AI

Azure AKS + Application Gateway Ingress Controller Deployment Context

Private networking, AKS microservices deployment, Azure AI Foundry, PostgreSQL + pgvector, Blob Storage, Service Bus, RabbitMQ, Key Vault, and production security design.

Field	Value
Project	ResolveOps AI
Repository	https://github.com/Sathvik307393/ResolveOps-AI.git
Current test domain	resolveops-ai.sathvikdevops.online
Target platform	Azure Kubernetes Service (AKS)
Public entry point	Azure Application Gateway WAF using AGIC
Security model	Private backend services, private endpoints, Key Vault, managed identities, RBAC
Deployment approach	Create manually in Azure Portal first, then convert to Terraform + CI/CD

1. Purpose of This Document

This document is the deployment context for moving ResolveOps AI from the current Docker Compose/EC2 style deployment into a secure Azure architecture. It lists the required Azure resources, explains how they connect, and provides step-by-step portal deployment guidance for an AKS-based deployment exposed through Application Gateway Ingress Controller (AGIC).

- Use this document as the starting point for a new Azure deployment chat or implementation session.
- The target is a production-style architecture but cost-aware for demo/presentation use.
- Backend resources must remain private. The only intended public entry point is Application Gateway WAF.
- The first phase is manual Azure Portal setup; the second phase is Terraform automation and CI/CD.

2. Application Overview

ResolveOps AI is an AI-powered autonomous SRE and incident intelligence platform. It connects to GitHub Actions, Azure, AWS, and Kubernetes environments to discover resources, read logs and metrics, analyze failures, generate RCA, and recommend fixes.

Module	Purpose
Cloud Resources	Central resource inventory across connected cloud providers.
GitHub Sync Hub	Synchronizes GitHub repositories, workflows, workflow runs, logs, and failed pipeline RCA.
Azure Hub	Discovers Azure resources, activity, cost, logs, risk context, and relationships.
AWS Hub	Discovers AWS EC2, RDS, EKS, S3, CloudWatch, cost, and risk details.
AI Copilot	Natural-language SRE assistant for incident and infrastructure queries.
Suggestions	Reliability, security, cost, and operational recommendations.
Analytics	High-level operational dashboard.
Integrations	Source of truth for connecting GitHub, Azure, AWS, and future tools.

3. Target Azure Architecture

The Azure target architecture follows a private-by-default model. Users reach the application through Application Gateway WAF. Application Gateway forwards traffic to AKS through AGIC. Microservices inside AKS access Azure PaaS resources over private endpoints and managed identities.

```
Internet Users
  |
  v
Azure Application Gateway WAF (public IP, TLS, path-based routing)
  |
  v
AGIC / AKS Ingress
  |
  +--> frontend-service
  +--> api-gateway-service
       |
       +--> auth-service
       +--> github-intelligence-service
       +--> azure-intelligence-service
       +--> aws-intelligence-service
       +--> ai-rca-service
       +--> cost-insights-service
       +--> notification-service
       +--> diagram-report-service
       |
       +--> PostgreSQL Flexible Server + pgvector (private)
       +--> Azure Blob Storage (private)
       +--> Azure Service Bus (private)
       +--> RabbitMQ inside AKS (internal)
       +--> Azure AI Foundry (private/secured)
       +--> Key Vault (private)
       +--> Azure Monitor / Log Analytics
```

4. Required Azure Resource List

Azure Resource	Suggested Name	Why It Is Needed
Resource Group	rg-resolveops-prod	Logical boundary for all production resources.
Virtual Network	vnet-resolveops-prod	Private network for AKS, Application Gateway, private endpoints, and Bastion.
Subnets	snet-appgateway, snet-aks, snet-private-endpoints, AzureBastionSubnet	Network segmentation.
AKS	aks-resolveops-prod	Runs frontend and backend microservices.
Application Gateway WAF	agw-resolveops-prod	Public HTTPS entry point with AGIC and path routing.
Azure Container Registry	acrresolveopsprod	Stores container images.
Key Vault	kv-resolveops-prod	Stores secrets, API keys, tokens, database passwords.
PostgreSQL Flexible Server	psql-resolveops-prod	Application database and pgvector embeddings.
Storage Account	stresolveopsprod	Stores RCA reports, diagrams, logs, and generated solution artifacts.
Service Bus	sb-resolveops-prod	Enterprise async event bus for scans, RCA, notifications, diagrams.
RabbitMQ	rabbitmq inside AKS	Internal task queue for workers and background jobs.
Azure AI Foundry	aif-resolveops-prod	Primary AI provider for RCA, Copilot, suggestions, diagram generation.
Log Analytics	law-resolveops-prod	Central observability workspace.

Application Insights	appi-resolveops-prod	Application telemetry and traces.
Azure Bastion	bas-resolveops-prod	Secure private troubleshooting access.
Private DNS Zones	privatelink.* zones	DNS resolution for private endpoints.
NAT Gateway or Firewall	nat-resolveops-prod / optional Azure Firewall	Controlled outbound access from private AKS if required.
Managed Identities	mi-resolveops-*	Workload identity and least-privilege access.

5. Network Design and Subnets

Use one hub VNet for the initial deployment. The subnets below keep ingress, AKS, private endpoints, and management separated.

Subnet	CIDR Example	Purpose	Important Notes
snet-appgateway	10.20.1.0/24	Dedicated subnet for Application Gateway WAF.	Application Gateway requires its own subnet.
snet-aks	10.20.2.0/23	AKS node subnet.	Use Azure CNI; size generously for pods/nodes.
snet-private-endpoints	10.20.4.0/24	Private endpoints for Key Vault, PostgreSQL, Storage, Service Bus, ACR.	Keep PaaS endpoints isolated.
AzureBastionSubnet	10.20.5.0/26	Azure Bastion.	Name must be exactly AzureBastionSubnet.
snet-management	10.20.6.0/24	Optional private VM/jumpbox.	Use only if needed; prefer Bastion.

- Application Gateway is public; AKS and all PaaS services should be private.
- Private endpoints should be placed in snet-private-endpoints.
- Private DNS zones must be linked to vnet-resolveops-prod.
- AKS egress should be controlled using NAT Gateway or Azure Firewall if strict outbound control is required.

6. AKS Deployment Design

AKS Setting	Recommended Value
Cluster name	aks-resolveops-prod
Network plugin	Azure CNI
Cluster type	Private cluster preferred
Identity	System-assigned or user-assigned managed identity
OIDC issuer	Enabled
Workload Identity	Enabled
Autoscaling	Enabled
Monitoring	Azure Monitor / Container Insights enabled
Ingress	Application Gateway Ingress Controller
RBAC	Entra ID integration + Kubernetes RBAC
Image source	Azure Container Registry

Node Pool	Purpose	Suggested Size	Autoscaling
systempool	System pods and core AKS services	Standard_D2s_v5 or similar	Min 2, Max 3
apppool	ResolveOps AI services	Standard_D4s_v5 or cost-aware equivalent	Min 2, Max 5
workerpool (optional)	RCA, diagram, log processing workers	Standard_D4s_v5 or memory optimized	Min 0/1, Max 5

7. Microservices to Deploy in AKS

Service	Kubernetes Name	Responsibilities
Frontend	frontend-service	Next.js/React UI for ResolveOps AI.
API Gateway	api-gateway-service	Single backend entry point; routes /api/* to internal services.

Auth Service	auth-service	Authentication, users, sessions, JWT handling.
GitHub Intelligence	github-intelligence-service	Repos, workflows, runs, logs, RCA data.
Azure Intelligence	azure-intelligence-service	Azure resource discovery, logs, costs, relationships.
AWS Intelligence	aws-intelligence-service	AWS resource discovery, metrics, CloudWatch, cost, risks.
AI RCA	ai-rca-service	Azure AI Foundry RCA, fallback analysis, Copilot intelligence.
Cost Insights	cost-insights-service	Cloud cost summaries and optimization hints.
Notification	notification-service	Email/notification delivery for incidents and RCA.
Diagram / Report	diagram-report-service	Diagram-as-code generation, report export, Blob upload.
RabbitMQ	rabbitmq	Internal async worker queue.
Kroki Renderer	kroki	Renders Mermaid/D2/PlantUML/Structurizr diagrams.

8. Application Gateway + AGIC Design

Application Gateway WAF will be the only public ingress. AGIC will watch Kubernetes Ingress resources and configure Application Gateway automatically.

Route	Backend in AKS	Purpose
/	frontend-service	ResolveOps AI UI
/api/*	api-gateway-service	All backend API calls
/health	api-gateway-service or frontend	Health probe endpoint
/metrics	internal only	Expose only internally or via monitoring

Ingress concept:

```
resolveops-ai.sathvikdevops.online/
  -> frontend-service:3000
resolveops-ai.sathvikdevops.online/api/*
  -> api-gateway-service:8000
```

- Use TLS certificate on Application Gateway.
- Use WAF policy in prevention or detection mode depending on demo maturity.
- Do not expose individual microservices publicly.
- All internal services should be ClusterIP.

9. Private Endpoints and Private DNS

Service	Private Endpoint Needed?	Private DNS Zone
Key Vault	Yes	privatelink.vaultcore.azure.net
Blob Storage	Yes	privatelink.blob.core.windows.net
PostgreSQL Flexible Server	Yes	privatelink.postgres.database.azure.com
Service Bus	Yes	privatelink.servicebus.windows.net
ACR	Recommended	privatelink.azurecr.io
Azure AI Foundry / AI service	Recommended where supported	Service-specific private link zone
Log Analytics	Optional/advanced	Azure Monitor Private Link Scope if required

- Create each private endpoint inside snet-private-endpoints.
- Disable public network access where supported after private connectivity is confirmed.
- Link all private DNS zones to vnet-resolveops-prod.
- Test DNS resolution from a pod or private VM before locking down public access.

10. Secrets and Configuration Strategy

Store all secrets in Azure Key Vault. AKS workloads should access secrets through Workload Identity and Key Vault CSI Driver or External Secrets Operator. Do not put secrets in Docker images, GitHub repository, frontend code, or plain ConfigMaps.

Secret	Where Stored	Used By
JWT_SECRET	Key Vault	Auth/API Gateway
DATABASE_URL / POSTGRES_PASSWORD	Key Vault	API/services needing DB
GITHUB_PAT / GitHub app secret	Key Vault	GitHub Intelligence
AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY	Key Vault temporarily for demo	AWS Intelligence
AZURE_CLIENT_SECRET	Key Vault only if service principal still used	Azure Intelligence
SENDGRID_API_KEY / SMTP_PASSWORD	Key Vault	Notification Service
AZURE_AI_FOUNDRY_API_KEY	Key Vault if key-based	AI RCA / Copilot
RABBITMQ_PASSWORD	Key Vault/Kubernetes secret backed by Key Vault	RabbitMQ clients
SERVICE_BUS_CONNECTION_STRING	Prefer identity; Key Vault if needed	Worker services

ConfigMap Value	Example
APP_NAME	ResolveOps AI
APP_ENV	prod
FRONTEND_URL	https://resolveops-ai.sathvikdevops.online
API_BASE_URL	https://resolveops-ai.sathvikdevops.online/api
AI_PROVIDER	azure_foundry
DEFAULT_AWS_REGION	us-east-1
BLOB_CONTAINER_REPORTS	reports
BLOB_CONTAINER_DIAGRAMS	diagrams
SERVICE_BUS_NAMESPACE	sb-resolveops-prod

11. Identity and RBAC Requirements

Identity	Required Roles / Access
AKS kubelet identity	AcrPull on ACR.
AGIC identity	Contributor on Application Gateway, Reader on resource group, Network Contributor if required.
api-gateway managed identity	Key Vault Secrets User, database access as needed.
github-intelligence identity	Read GitHub secret from Key Vault; DB read/write.
azure-intelligence identity	Reader, Monitoring Reader, Cost Management Reader on target subscription/resource groups.
aws-intelligence identity	Read AWS credentials from Key Vault during demo; later use external role flow.
ai-rca identity	Access Azure AI Foundry, Blob reports, database RCA tables.
diagram-report identity	Blob Data Contributor for diagrams/reports, access to AI Foundry and renderer.
notification identity	Read SMTP/SendGrid secret from Key Vault.

12. Data and Messaging Design

Data / Messaging Resource	Use in ResolveOps AI
PostgreSQL + pgvector	Users, integrations, resource inventory, workflow metadata, RCA reports, embeddings, suggestions.
Blob Storage	RCA report files, raw/sanitized log snapshots, generated solution documents, architecture diagrams.
Service Bus	Durable async events: resource scans, RCA requests, notifications, diagram generation.
RabbitMQ	Fast internal worker jobs inside AKS, such as log parsing and report rendering.
Azure AI Foundry	Primary intelligence layer for RCA, Copilot, suggestions, and diagram code generation.

Service Bus Queue / Topic	Purpose
github-sync-requested	Start GitHub repo/workflow/run sync.
aws-sync-requested	Start AWS resource scan.
azure-sync-requested	Start Azure resource scan.
resource-scan-completed	Notify UI/analytics after scan.
rca-requested	Trigger RCA worker.
rca-completed	Notify UI/notification service after RCA.
notification-requested	Send email or other notification.
diagram-generation-requested	Generate diagram-as-code and render artifact.

13. Azure AI Foundry and RCA Flow

Failed workflow / incident selected
|
v
Collect logs, events, metrics, resource metadata
|
v
Redact secrets and sensitive values
|
v
Retrieve similar incidents from PostgreSQL + pgvector
|
v
Send structured context to Azure AI Foundry
|
v
Generate RCA: summary, root cause, evidence, fix steps, confidence
|
v
Store metadata in PostgreSQL and report artifact in Blob Storage

- Azure AI Foundry should be the primary AI provider in Azure production.
- Bedrock can remain optional but should not be the only RCA provider.
- Rule-based RCA fallback should generate useful output even when the AI model is unavailable.
- All logs must be redacted before being shown to users or sent to AI.

14. Manual Azure Portal Deployment Steps

Use these steps in order. The goal is to get the Azure foundation working manually before converting it into Terraform.

1. Create Resource Group: rg-resolveops-prod in the chosen primary Azure region.
2. Create VNet vnet-resolveops-prod with subnets: snet-appgateway, snet-aks, snet-private-endpoints, AzureBastionSubnet.
3. Create Log Analytics Workspace law-resolveops-prod.
4. Create Azure Container Registry acrresolveopsprod and plan private endpoint / AKS AcrPull access.
5. Create Key Vault kv-resolveops-prod with RBAC access model and private endpoint.
6. Create PostgreSQL Flexible Server psql-resolveops-prod with private networking and pgvector enabled.
7. Create Storage Account stresolveopsprod, disable public blob access, create containers reports, diagrams, logs, solutions.
8. Create Service Bus namespace sb-resolveops-prod and required queues/topics.
9. Create Azure AI Foundry / AI project resources and store endpoint/key/deployment info securely.
10. Create AKS aks-resolveops-prod with Azure CNI, Workload Identity, OIDC, autoscaler, monitoring, ACR integration.
11. Create Application Gateway WAF agw-resolveops-prod in snet-appgateway.
12. Enable/configure AGIC for AKS and Application Gateway.

13. Create private endpoints and private DNS zones for Key Vault, Storage, PostgreSQL, Service Bus, and ACR.
14. Create Azure Bastion for private troubleshooting access.
15. Build Docker images and push to ACR.
16. Create Kubernetes namespaces and deploy base components: RabbitMQ, Kroki, app ConfigMaps, Key Vault CSI/External Secrets.
17. Deploy ResolveOps AI microservices to AKS.
18. Create Kubernetes Ingress for Application Gateway routing.
19. Configure DNS for resolveops-ai.sathvikdevops.online to Application Gateway public IP.
20. Configure TLS certificate on Application Gateway.
21. Validate the app end-to-end.

15. Kubernetes Deployment Checklist

Kubernetes Object	Needed For
Namespaces	resolveops-app, resolveops-system, resolveops-observability
Deployments	One per microservice.
Services	ClusterIP for internal services; frontend/API Gateway behind ingress.
Ingress	AGIC path-based routing.
ConfigMaps	Non-sensitive app configuration.
Secrets / SecretProviderClass	Key Vault-backed secret injection.
ServiceAccounts	Workload Identity per service.
HPA	Autoscaling for API, frontend, workers.
NetworkPolicies	Restrict service-to-service traffic.
Probes	Readiness/liveness for each service.
PodDisruptionBudgets	Availability during maintenance.

Suggested namespaces:

```
kubectl create namespace resolveops-app
kubectl create namespace resolveops-system
kubectl create namespace resolveops-observability
```

16. Validation Checklist

Validation	Expected Result
Application Gateway public IP opens domain	Frontend loads over HTTPS.
/api health route	API Gateway responds successfully.
AKS pods	All deployments Ready.
ACR pull	AKS pulls images without image pull errors.
Key Vault secret mount	Pods receive required secrets without exposing them.
PostgreSQL connectivity	API Gateway/auth/services connect privately.
Blob write test	RCA/report/diagram container write succeeds.
Service Bus send/receive	Workers can process messages.
RabbitMQ internal access	Workers connect using internal service DNS.
Azure AI Foundry RCA test	AI RCA or fallback RCA works.
GitHub integration	PAT sync works and repos/workflows/runs show.
AWS integration	Connection/status/sync works after credentials setup.
Azure Hub	Resource discovery and logs work.
Monitoring	AKS and app logs visible in Log Analytics.

Useful validation commands:

```
kubectl get nodes
kubectl get pods -A
kubectl get ingress -A
kubectl get svc -A
kubectl describe ingress -n resolveops-app
kubectl logs deploy/api-gateway-service -n resolveops-app --tail=100
kubectl logs deploy/frontend-service -n resolveops-app --tail=100
```

17. Security Checklist

- Only Application Gateway should be public.
- AKS API server should be private or restricted.
- Use private endpoints for PostgreSQL, Storage, Key Vault, Service Bus, and ACR where possible.
- Disable public network access for PaaS services after private connectivity is confirmed.
- Use Key Vault for all secrets.
- Use Workload Identity for AKS service access.
- Do not store GitHub PAT, AWS keys, Azure secrets, or SMTP keys in frontend code or GitHub repo.
- Rotate any AWS access keys exposed during screenshots/testing.
- Enable diagnostic settings for Application Gateway, AKS, Key Vault, PostgreSQL, Storage, and Service Bus.
- Use WAF on Application Gateway.
- Use least-privilege RBAC for all managed identities.
- Enable alerts for pod crashes, App Gateway 5xx, DB issues, Key Vault failures, and Service Bus dead letters.

18. High Availability and Scalability Plan

Layer	HA / Scalability Approach
Application Gateway	Use WAF v2 with autoscaling and zone redundancy if supported.
AKS	Multiple nodes, availability zones, cluster autoscaler, HPA.
Microservices	Run at least 2 replicas for critical services in production.
PostgreSQL	Zone redundant HA where cost allows; backups enabled.
Storage	ZRS or GRS depending on required durability.
Service Bus	Premium tier for private endpoint and predictable throughput if production-grade.
RabbitMQ	Clustered RabbitMQ for production; single instance acceptable for demo.
AI/RCA Workers	Queue-based workers scale independently.
Multi-region	Deploy secondary region later with replicated storage/database strategy and Traffic Manager/Front Door if required.

19. Terraform and CI/CD Phase After Manual Deployment

- After portal deployment is validated, convert infrastructure to Terraform modules.
- Use Azure Storage backend for Terraform state and state locking.
- Use GitHub Actions OIDC to Azure instead of long-lived client secrets.
- Add Checkov for IaC security scanning.
- Application pipeline should build images, run tests, scan with SonarQube/Snyk/Trivy, push to ACR, and deploy through ArgoCD or Helm.
- DAST scan should run after deployment.
- Keep dev/stage/prod separated through workspaces or environment-specific variable files.

20. Recommended Azure Resource Creation Order

Order	Resource	Reason
1	Resource Group	Foundation for all resources.
2	VNet/Subnets	Required for private resources.
3	Log Analytics	Attach monitoring during creation.
4	ACR	Needed before app image deployment.
5	Key Vault	Store secrets early.
6	PostgreSQL	Core database.
7	Storage Account	Reports/logs/diagrams.
8	Service Bus	Async workflows.
9	Azure AI Foundry	AI RCA and Copilot.
10	AKS	Application compute platform.
11	Application Gateway WAF	Public ingress.
12	AGIC	Connect App Gateway to AKS Ingress.
13	Private Endpoints/DNS	Lock down PaaS networking.

14	Bastion	Private troubleshooting.
15	Deploy app	Deploy and validate microservices.
16	DNS/TLS	Production domain and HTTPS.
17	Alerts/Monitoring	Operational readiness.
18	Terraform	Automate after manual success.

21. Key Decisions to Keep Consistent

- Deploy on AKS because the app has multiple microservices, RabbitMQ, workers, and future renderer pods.
- Use Application Gateway WAF + AGIC for ingress instead of exposing AKS services directly.
- Keep Integrations page as the single source of truth for GitHub/Azure/AWS authentication.
- Keep all backend services private and internal.
- Use Azure AI Foundry as the primary RCA/AI provider in Azure.
- Use PostgreSQL + pgvector for application data and AI retrieval memory.
- Use Blob Storage for generated artifacts such as reports, logs, solutions, and diagrams.
- Use Service Bus for durable events and RabbitMQ for internal worker queues.
- Do manual portal setup first; then generate Terraform and pipelines.

22. Starting Point for the Next Chat

Use the following instruction in the new chat:

Using the attached ResolveOps AI Azure AKS + AGIC Deployment Context,
help me create the Azure resources manually through the Azure Portal step by step.
Start with Resource Group, VNet, subnets, and networking. Keep all backend resources private.
The app will be deployed in AKS and exposed through Application Gateway Ingress Controller.

End of Deployment Context